

INFORME TÉCNICO: ESTRATEGIA Y PLANIFICACIÓN DE PRUEBAS DE SOFTWARE

PROYECTO: SISTEMA MAYORO — EVALUACIÓN PARCIAL

DATOS GENERALES

- **Curso:** Pruebas y Calidad de Software
- **Proyecto:** Sistema MAYORO
- **Integrantes del Equipo:**
 - Manuel Fabrizio Callañaupa Cjuiro — Product Owner/Revisor de Cambios
 - Carlos Eduardo Loaiza Martinez — Scrum master, Senior QA
 - Rodrigo Eduardo Cussi Valenzuela — Responsable Técnico de Construcción
 - Paul Gonzalo Acurio Jara — Coordinador de Configuración
- **Repositorio de Código y Pruebas:** <https://github.com/MayorosCorp/Mayoro>

1. CONTEXTO DEL PROYECTO Y ALCANCE DE PRUEBAS

1.1 Descripción y Propósito del Sistema MAYORO

El Sistema MAYORO es una plataforma web B2B orientada a digitalizar y optimizar la cadena de suministro comercial entre comerciantes minoristas (bodegas) y distribuidores mayoristas. Su propósito principal es eliminar la fricción operativa del aprovisionamiento tradicional mediante la centralización del catálogo mayorista, la aplicación de condiciones mínimas de venta por lote (MOQ) y la derivación automatizada de cotizaciones formales estructuradas hacia los canales de comunicación directa de los proveedores (WhatsApp).

El sistema resuelve problemáticas críticas de negocio:

- Informalidad e inconsistencias en la validación fiscal de empresas participantes (control de RUC de 11 dígitos y duplicidad de credenciales).
- Publicación desordenada de productos sin reglas de empaque o volumen mínimo.
- Errores de cálculo en subtotales, recargo del Impuesto General a las Ventas (IGV) y pedidos que no alcanzan el umbral de venta mayorista.

1.2 Funcionalidades Principales

El proyecto está implementado bajo un patrón arquitectónico desacoplado y estructurado en Laravel (PHP 8.2+) sobre el motor relacional MySQL (bases de datos `mayoro\_db` para desarrollo local y `mayoro\_db\_testing` para ejecución aislada de pruebas). La capa de presentación utiliza plantillas dinámicas Blade combinadas con Tailwind CSS.

Actualmente, el sistema se encuentra en un nivel de madurez funcional Básico/Intermedio: las capas de persistencia (migraciones y modelos Eloquent), los controladores de autenticación, el servicio de catálogo mayorista y el motor transaccional de cotizaciones vía WhatsApp (`WhatsAppService`) están completamente implementados e integrados en la rama de integración `develop`, respaldados por una suite de pruebas automatizadas en PHPUnit y gestionados a través de sprints en Jira.

1.3 Estado Actual de Desarrollo

El proyecto se encuentra en un nivel de madurez BÁSICO-INTERMEDIO. La arquitectura está construida sobre Laravel (PHP 8.2+) utilizando MySQL como motor de base de datos (`mayoro\_db` para desarrollo local y `mayoro\_db\_testing` para el entorno de pruebas), y vistas dinámicas en Blade con Tailwind CSS.

1.4 Delimitación del Alcance de Pruebas

Módulos y Componentes INCLUIDOS:

- Modelos de Datos Eloquent y Esquemas de Base de Datos: Verificación de atributos,

restricciones de unicidad, reglas de longitud (RUC de 11 dígitos, teléfonos en formato numérico) y conversiones numéricas/castings en modelos `User`/`UsuarioB2B`, `Product`/`ProductoMayorista` y `Quote`/`CotizacionB2B`.

- Reglas de Negocio en Servicios Core: Validación unitaria de `WhatsAppService` (generación sintáctica del enlace, codificación RFC 3986 de caracteres especiales, excepción ante teléfonos inválidos o cantidades menores o iguales a cero).
- Regla de Negocio Transaccional de Pedido Mínimo: Verificación de que ninguna cotización proceda si la cantidad solicitada no cumple la restricción `cantidad >= MOQ`.
- Pruebas de Aislamiento con Dobles de Prueba: Empleo de Mockery sobre interfaces externas para certificar el desacoplamiento de la lógica de cotizaciones frente a clientes externos.

- Módulos y Aspectos EXCLUIDOS:

- Pruebas de Carga y Estrés de Concurrencia: Excluidas técnicamente porque la infraestructura actual corresponde a entornos locales de desarrollo (XAMPP/Laragon); la evaluación de concurrencia masiva se ejecutará en la etapa final con el entorno staging configurado.
- Módulo Premium y Pasarela Transaccional de Pagos (HU-05): Excluido del alcance de pruebas de esta entrega debido a que se encuentra en estado de refinamiento funcional e investigación de sandbox bancario (planificado para el Sprint 3).
- Pruebas de Penetración y Vulnerabilidades Dinámicas (DAST): Se prioriza la calidad funcional, la integridad de datos del ORM y la lógica de negocio; el análisis de penetración integral se reserva para el release final pre-producción.

2. ESTRATEGIA Y PLANIFICACIÓN DE PRUEBAS

2.1 Tipos y Niveles de Pruebas Aplicados

En la arquitectura del Sistema MAYORO (desarrollado sobre Laravel 12 con PHP 8.2+ y MySQL), la estrategia de aseguramiento de la calidad combina un enfoque basado en riesgos y alineado al Modelo V. A continuación, se detallan los tipos y niveles de prueba implementados en el proyecto:

Nivel de Prueba	Categoría / Tipo	Componente Evaluado	Propósito y Justificación Técnica
-----------------	------------------	---------------------	-----------------------------------

Componente (Unitaria)	Funcional	WhatsAppService WaMeLinkGeneratorTest	/	Valida de forma aislada la lógica de negocio para la generación del enlace codificado de WhatsApp (urlencode), verificando el formateo de mensajes y la captura de excepciones ante teléfonos inválidos o cantidades no válidas.
Componente (Unitaria)	No Funcional (Seguridad / Robustez)	ProductImageStorageTest		Evalúa el saneamiento de nombres de archivos, truncado de textos extensos a 60 caracteres y la prevención de vulnerabilidades de Path Traversal al eliminar archivos.
Componente (Unitaria)	Funcional	QuotationTotalsTest QuotationServiceTest	y	Verifica el cálculo exacto de subtotales, discriminación del IGV (18 %) y el cumplimiento estricto del Pedido Mínimo o Minimum Order Quantity (MOQ).
Integración / BDD	Funcional	UserRegistrationB2BTest		Comprueba la interacción entre formularios, controladores y la base de datos users_b2b, garantizando la regla regex de 11 dígitos para el RUC y el control de duplicados.
Integración / BDD	Funcional	ProductStockStateTest		Valida la coherencia de la matriz de estados del catálogo (Disponible, Sin Stock, Pausado) ante variaciones en el inventario real y la visibilidad para

			compradores.
Integración / BDD	No Funcional (Seguridad / RBAC)	ProductPublishingNavigationTest	Verifica el control de acceso basado en roles (RBAC) durante la navegación y publicación de productos, asegurando que únicamente los usuarios con permisos correspondientes puedan acceder a las funcionalidades restringidas.

2.2 Riesgos y Errores Críticos a Mitigar

Se estructuran los principales riesgos del negocio B2B, su nivel de impacto y el mecanismo técnico aplicado para prevenirlos o mitigarlos:

Nivel de Riesgo	Riesgo / Error Crítico	Impacto en el Sistema	Mecanismo de Mitigación / Prueba Aplicada
Crítico	Duplicidad o Formato Inválido en Datos Fiscales (RUC/Email)	Registro de empresas no legítimas o colisión de identidades comerciales en la base de datos.	Pruebas de integración en UserRegistrationB2BTest con validación estricta de expresiones regulares de 11 dígitos y restricción de unicidad en BD.
Crítico	Vulnerabilidades de Seguridad Web (XSS, SQLi y Path Traversal)	Ejecución de scripts maliciosos en formularios o eliminación no autorizada de archivos del servidor.	Sanitización obligatoria en Form Requests y pruebas unitarias de almacenamiento en ProductImageStorageTest para bloquear secuencias de escape (..).

Alto	Violación de Reglas Comerciales (Precios Negativos o < MOQ)	Generación de cotizaciones inválidas por debajo del lote mínimo configurado por el distribuidor.	Validaciones numéricas en QuotationServiceTest y QuoteWhatsAppFlowTest que bloquean la acción de envío si la cantidad es menor al MOQ.
Alto	Inseguridad en Control de Acceso (IDOR y Bypass de Roles)	Un distribuidor manipula la URL para editar o alterar el catálogo de otra empresa comercial.	Control de acceso basado en roles (RBAC) evaluado mediante ProductPublishingNavigationTest, devolviendo error HTTP 403 ante peticiones cruzadas.
Medio	Inconsistencia en la Matriz de Stock (Inexistencia o Pausa Irreversible)	Productos que nacen como "Sin stock" de forma predeterminada o quedan inalcanzables al pausarlos.	Cobertura integral en ProductStockStateTest sobre la combinación de banderas de disponibilidad y la columna stock_disponible.

2.3 Priorización de Componentes (backlog)

La siguiente tabla define la prioridad de ejecución y verificación de los componentes del sistema:

Prioridad	Componente / Modelo	Justificación Técnica
Alta	Módulo de Autenticación y Registro B2B (HU-01)	Crítico para la operación legal del negocio B2B. Garantiza la validez fiscal (RUC de 11 dígitos), unicidad de correos y la seguridad en el hashing de contraseñas mediante Bcrypt.
Alta	Gestión de Catálogo Mayorista y Stock (HU-02 / HU-03)	Determina la integridad de los datos financieros (precios, montos e IGV) y restringe mediante RBAC que usuarios no autorizados alteren el catálogo.
Media	Servicio de Cotizaciones por WhatsApp (HU-04)	Representa una funcionalidad transaccional principal de la plataforma. Verifica el cumplimiento del MOQ y la construcción correcta del enlace codificado para la comunicación comercial.

Baja	Módulo SaaS de Inventario Premium (HU-05)	Funcionalidad diferencial orientada a la monetización. Se evalúa al final del ciclo por ser una extensión para cuentas de pago protegida mediante middleware.
------	---	---

2.4 Criterios de Aceptación y Salida

Para dar por aprobada la suite de pruebas del proyecto MAYORO, se deben cumplir los siguientes criterios técnicos:

1. Criterios de Entrada:

- Código fuente sincronizado en la rama de integración **develop** sin conflictos.
- Entorno local parametrizado con la base de datos SQLite en memoria (**RefreshDatabase**).

2. Criterios de Salida:

- **Porcentaje de Aprobación del 100%:** Cero fallos (**0 FAILED**) en la suite completa de 133 pruebas y 399 aserciones.
- **Defectos Críticos Corregidos:** Verificación y cierre de los 6 defectos detectados durante el desarrollo (incluyendo correcciones de login, IDOR y Path Traversal).
- **Cumplimiento de Estilo:** Pasar la verificación automatizada del estándar PSR-12 mediante Laravel Pint.

2.5 Matriz de Roles y Distribución de Responsabilidades

Integrante Asignado	Módulo / Modelo Asignado	Tipo de Prueba	Fecha Límite
Paul Gonzalo Acurio Jara	Módulo de Registro y Configuración SCM (HU-01)	Pruebas de Integración y Scripts SCM	25/09/2026
Rodrigo Eduardo Cussi Valenzuela	Cotizaciones por WhatsApp e Imágenes (HU-04, HU-02)	Pruebas Unitarias TDD y Mocks	27/09/2026
Manuel Fabricio Callañaupa Cjuiro	Estado de Stock y Publicación Mayorista (HU-02, HU-03)	Pruebas BDD / Feature / Integration	28/09/2026
Carlos Eduardo Loaiza Martinez	Directorio de Proveedores y Módulo Premium (HU-03, HU-05)	Pruebas de Integración y Trazabilidad	29/09/2026

Reporte de todas las pruebas realizadas en el proyecto 06/10/2026

https://docs.google.com/document/d/1f_I8rLEPhc3DNDBTs8Vhgd1npBNCcAtt/edit?usp=sharing&ouid=100407227444266904753&rtpof=true&sd=true

3. DISEÑO E IMPLEMENTACIÓN DE PRUEBAS

3.1 Pruebas de Modelo

Prueba M01: Registro Comercial B2B con Validación de RUC (HU-01)

- Comportamiento / Regla a validar: Verificar que el sistema registre exitosamente a una empresa cuando el RUC posee exactamente 11 dígitos numéricos, el correo es único y sanitice las entradas contra ataques XSS/SQLi. Rechazar solicitudes con RUCs duplicados.
- Datos de entrada: Array con `ruc_empresa = "20601234567"`, `email_contacto = "contacto@bodega.pe"`, `razon_social = "Bodega Central S.A.C."`, `telefono_whatsapp = "984123456"`.
- Resultado esperado: Respuesta HTTP 302/200 con la entidad guardada en la base de datos `users_b2b` y rechazo HTTP de validación si el RUC posee 10 dígitos o está duplicado.

Código de la prueba implementada:

```
/**
 * Test para verificar el registro exitoso de un comercio B2B y rechazo de RUCs
 * duplicados.
 */
public function test_registro_usuario_b2b_exitoso_y_control_de_duplicados(): void
{
    $userData = [
        'ruc_empresa' => '20601234567',
        'razon_social' => 'Bodega Comercial Cusco S.A.C.',
        'email_contacto' => 'admin@bodegacusco.pe',
        'password' => 'Password123!',
        'telefono_whatsapp' => '984123456',
        'rol' => 'bodega'
    ];

    $response = $this->post('/register', $userData);
    $response->assertStatus(302);
    $this->assertDatabaseHas('users_b2b', ['ruc_empresa' => '20601234567']);

    // Intento de duplicado
    $duplicateResponse = $this->post('/register', $userData);
    $duplicateResponse->assertSessionHasErrors(['ruc_empresa', 'email_contacto']);
}
```

```
}
```

Sustento técnico: Previene la suplantación de identidad fiscal en la red B2B, garantiza la consistencia de datos según la normativa impositiva peruana y mitiga accesos indebidos

Prueba M02: Matriz de Estados de Disponibilidad de Stock (HU-03 / ProductStockStateTest)

- Comportamiento / Regla a validar: Comprobar que un producto altere su visibilidad e indicación en el catálogo público según las combinaciones de las banderas **activo** y **stock_disponible** (disponible, sin stock o pausado por el distribuidor).
- Datos de entrada: Fixture de producto con **precio_bulto** = 78.00, **moq_cantidad_minima** = 5, variando **stock_disponible** = 0 y **estado** = "pausado".
- Resultado esperado: La vista renderizada contiene las etiquetas exactas "Disponible", "Temporalmente sin stock" o "Pausado por el distribuidor", desactivando la cotización cuando no es adquirible.
- Código de la prueba implementada:

```
/**
 * Test de integración para validar las etiquetas visuales del catálogo según el stock.
 */
public function test_evalua_matriz_de_estados_de_stock_en_catalogo(): void
{
    $distribuidor = User::factory()->create(['rol' => 'distribuidor']);
    $producto = Product::factory()->create([
        'user_id' => $distribuidor->id,
        'stock_disponible' => 0,
        'activo' => true
    ]);

    $response = $this->get('/products/' . $producto->id);
    $response->assertStatus(200);
    $response->assertSee('Temporalmente sin stock');
    $response->assertDontSee('Cotizar Pedido');
}
```

Prueba U01: Construcción de URL Codificada de Cotización para WhatsApp (WhatsAppService)

- Comportamiento validado: Evalúa la lógica pura del servicio encargada de sanear el número telefónico, validar que la cantidad cumpla con la restricción positiva y retornar la URL de la API de WhatsApp (<https://wa.me/51>...) con

el mensaje codificado mediante `urlencode()`.

- Casos de prueba:
 1. *Camino normal*: Teléfono válido, producto y montos correctos genera la URL codificada.
 2. *Excepción*: Teléfono con menos de 9 dígitos o cantidad solicitada ≤ 0 debe lanzar `InvalidArgumentException`.
- Código de la prueba unitaria:

```
<?php
namespace Tests\Unit;

use App\Services\WhatsAppService;
use InvalidArgumentException;
use PHPUnit\Framework\TestCase;

class ConditionWhatsAppTest extends TestCase
{
    public function test_genera_enlace_de_whatsapp_con_parametros_validos(): void
    {
        $service = new WhatsAppService();
        $phone = "984123456";
        $product = "Saco de Azucar 50kg";
        $quantity = 10;
        $totalPrice = 1450.00;

        $url = service->buildUrl(phone, $product, $quantity, $totalPrice);

        $this->assertStringStartsWith('https://wa.me/51984123456?text=', $url);
        $this->assertStringContainsString(urlencode('Saco de Azucar 50kg'), $url);
    }

    public function test_lanza_excepcion_si_el_telefono_es_invalido(): void
    {
        $service = new WhatsAppService();
        $this->expectException(InvalidArgumentException::class);
        $this->expectExceptionMessage('El numero de telefono es invalido.');
```

Alineación con la estrategia: Asegura que la redirección a la pasarela externa de WhatsApp nunca se rompa por caracteres especiales (espacios, saltos de línea, símbolos de moneda)

Prueba U02: Sanitización y Control de Nombres en Almacenamiento de Imágenes (ProductImageStorageTest)

- Comportamiento validado: Comprueba que la clase encargada del manejo de archivos sanitice nombres extensos (máximo 60 caracteres), prevenga la sobrescritura de imágenes existentes, reemplace extensiones falsas (ej. `.png` renombrado como `.pdf`) y bloquee ataques de Path Traversal al eliminar archivos.
- Código de la prueba unitaria:

```
/**
 * Test para validar la prevención de ataques Path Traversal al eliminar imágenes de
 * producto.
 */
public function test_previene_excepciones_no_controladas_por_path_traversal(): void
{
    Storage::fake('public');
    $storageService = new ProductImageStorageService();

    // Intento de borrado fuera del directorio 'productos' usando la secuencia '..'
    $pathMalicioso = '../evaluacion_sistema/archivo_critico.php';

    $resultado = storageService->eliminar(pathMalicioso);

    this->assertFalse(resultado);
    Storage::disk('public')->assertMissing($pathMalicioso);
}
```

Alineación con la estrategia: Elimina vulnerabilidades de seguridad en el servidor de archivos (subida arbitraria y eliminación de archivos fuera del directorio permitido), corrigiendo un defecto que causaba errores HTTP 500

4. RESULTADOS, ANÁLISIS TÉCNICO Y CONCLUSIONES

4.1 Reporte de Ejecución de Pruebas

Resumen de resultados tras la ejecución de las pruebas implementadas:

Identifica dor	Historia de Usuario	Tipo de Prueba	Componente Evaluado	Resulta do	Tiempo de Ejecuci ón
U01	HU-02 (Imágenes)	Prueba Unitaria	ProductImageStorageTest.php (23 tests)	PASSE D	0.12 s

U02	HU-04 (WhatsApp)	Prueba Unitaria	WaMeLinkGeneratorTest.php (5 tests)	PASSED	0.04 s
U03	HU-04 (Cotizaciones)	Prueba Unitaria	QuotationTotalsTest.php (3 tests)	PASSED	0.02 s
U04	HU-04 (Cotizaciones)	Prueba Unitaria	QuotationServiceTest.php (3 tests)	PASSED	0.03 s
U05	HU-04 (WhatsApp)	Prueba Unitaria	QuotationWhatsAppTest.php (1 test)	PASSED	0.01 s
M01	HU-01 (Autenticación)	Integración / BDD	UserRegistrationB2BTest.php (4 tests)	PASSED	0.19 s
M02	HU-01 (Autenticación)	Integración / BDD	B2BLoginTest.php (4 tests)	PASSED	0.15 s
M03	HU-02 (Publicación)	Integración / BDD	ProductPublishingB2BTest.php (5 tests)	PASSED	0.22 s
M04	HU-02 (Navegación RBAC)	Integración / BDD	ProductPublishingNavigationTest.php (13 tests)	PASSED	0.31 s
M05	HU-02 (Imágenes BDD)	Integración / BDD	ProductImageUploadTest.php (8 tests)	PASSED	0.25 s
M06	HU-03 (Catálogo)	Integración / BDD	PublicCatalogTest.php (16 tests)	PASSED	0.38 s
M07	HU-03 (Proveedores)	Integración / BDD	SupplierDirectoryTest.php (6 tests)	PASSED	0.18 s
M08	HU-03 (Modal Login)	Integración / BDD	QuoteModalTest.php (9 tests)	PASSED	0.21 s
M09	HU-03 (Stock Catálogo)	Integración / BDD	ProductStockStateTest.php (23 tests)	PASSED	0.45 s
M10	HU-04 (Flujo WhatsApp)	Integración / BDD	QuoteWhatsAppFlowTest.php (8 tests)	PASSED	0.28 s

Evidencia de Consola del Test Runner:

[Insertar aquí la captura de pantalla o el reporte de salida en texto obtenido al ejecutar la suite de pruebas en consola]

4.2 Interpretación Técnica de Resultados

Tras la ejecución integral de la suite se obtuvieron los siguientes hallazgos y correcciones aplicadas:

1. **Resolución de Colisión en Namepaces:** Se identificó una discrepancia inicial donde el conteo marcaba 135 pruebas. El análisis técnico detectó una colisión de nombres provocada por la presencia de dos archivos `ExampleTest` en `tests/Unit` y `tests/Feature`. Se corrigió aislando las ejecuciones por ruta.
2. **Corrección de Defectos Críticos Detectados durante el Testing:**
 - *Corrección en Login B2B:* El controlador realizaba la autenticación consultando la columna `email` en lugar de `email contacto`. La prueba `B2BLoginTest` detectó el fallo antes de pasar a producción.
 - *Omisión de Stock Inicial:* La acción `ProductController@store` no asignaba la columna `stock_disponible`. Fue subsanado en `ProductStockStateTes`.
 - *Manejo de Excepciones Path Traversal:* La eliminación de imágenes con la secuencia `..` provocaba una excepción no controlada resultando en HTTP 500. Se reestructuró la lógica para capturar la ruta de forma segura y retornar un estado booleano.
 - *Parche a Edición Cruzada (IDOR):* La prueba `ProductPublishingNavigationTest` confirmó el bloqueo con código HTTP 403 cuando un distribuidor intentaba modificar productos pertenecientes a otro usuario

4.3 Conclusiones y Retrospectiva

- **Utilidad de la Estrategia:** La combinación del enfoque TDD en la capa de servicios unitarios con pruebas de integración BDD sobre Laravel redujo el margen de error en un 100% para las funciones transaccionales del proyecto MAYORO.
- **Dificultades Encontradas:** El principal reto técnico residió en simular las peticiones externas hacia la API de WhatsApp y el sistema de archivos sin depender de servicios de terceros en la ejecución de los pipelines. Esto se superó integrando dobles de prueba con la librería `Mockery` y habilitando el disco ficticio `Storage::fake()`.
- **Mejoras para la Siguiente Iteración:**
 1. Integrar las herramientas `PCOV` o `Xdebug` en la canalización de CI/CD para medir automáticamente el porcentaje exacto de cobertura de código por líneas y ramas.
 2. Implementar pruebas E2E verdaderas sobre el navegador utilizando `Laravel Dusk` o `Playwright` para validar las interacciones JS en los modales de

cotización.

3. Añadir mecanismos de bloqueo optimista (*optimistic locking*) para evaluar la concurrencia de pedidos simultáneos sobre el stock disponible